

OccupiedLength [SimTalk] - lane A or B

Returns the part of the entire length of the specified lane of the *TwoLaneTrack* designated by <Path> which is occupied by all *Transporters* located on it.

Remarks

Each *Transporter* located on the *TwoLaneTrack* occupies part of the entire available length on the *TwoLaneTrack*.

Type

Read-only attribute

Syntax

```
<Path>.A.OccupiedLength → length  
<Path>.B.OccupiedLength → length
```

Return Value

The return value has the data type *length*.

Example

```
print myTwoLaneTrack.A.OccupiedLength
```

See also

[Length \[text box\] - lane A](#)

[Length \[text box\] - lane B](#)

Attributes of the TwoLaneTrack

_Attributes of the TwoLaneTrack

The *TwoLaneTrack* provides:

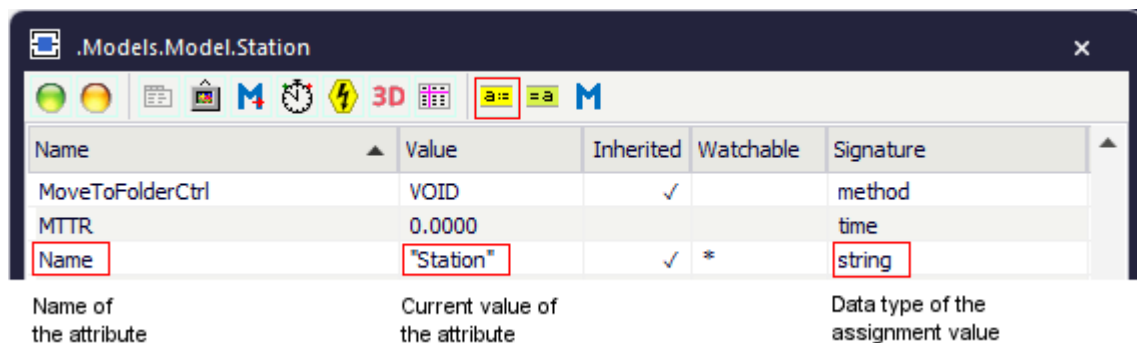
- The *attributes* listed in the table of contents to the left.
- The [Attributes of All Objects](#).
- The [Attributes of the Material Flow Objects](#).

Note

A number of attributes, which the *TwoLaneTrack* shares with the other material flow objects, apply to a lane, instead of to the entire object.

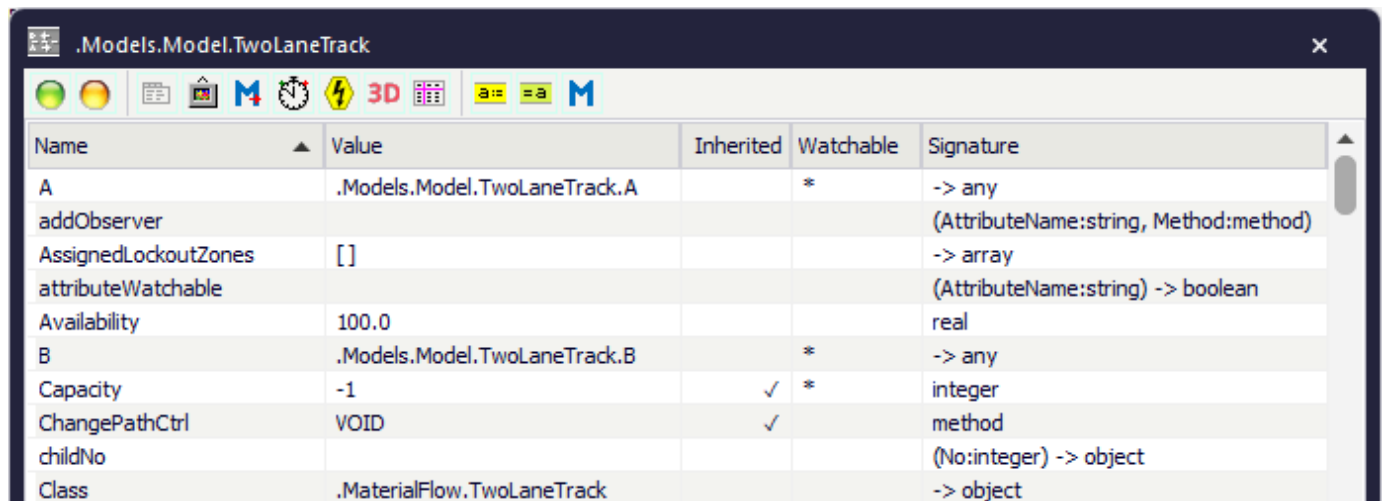
To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.

For the *TwoLaneTrack* the dialog shows **A** and **B** for the respective lane, but not the *attributes* proper for these lanes.



Name	Value	Inherited	Watchable	Signature
MoveToFolderCtrl	VOID	✓		method
MTTR	0.0000			time
Name	"Station"	✓	*	string

Name of the attribute Current value of the attribute Data type of the assignment value



Name	Value	Inherited	Watchable	Signature
A	.Models.Model.TwoLaneTrack.A		*	-> any
addObserver				(AttributeName:string, Method:method)
AssignedLockoutZones	[]			-> array
attributeWatchable				(AttributeName:string) -> boolean
Availability	100.0			real
B	.Models.Model.TwoLaneTrack.B		*	-> any
Capacity	-1	✓	*	integer
ChangePathCtrl	VOID	✓		method
childNo				(No:integer) -> object
Class	.MaterialFlow.TwoLaneTrack			-> object

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class** [general description].
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance** [general description].

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyTwoLaneTrack.B.Length:= 44
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyTwoLaneTrack.B.Length  
posit := MyStation.Cont.XPos
```

See also

[_Methods of the TwoLaneTrack](#)

[_Read-Only Attributes of the TwoLaneTrack](#)

BwEntranceCtrl [SimTalk] - lane A or B

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when a *Transporter* enters the specified lane of the *TwoLaneTrack* designated by <Path> via its exit and/or when the *Transporter* moves toward its entrance after entering.

Type

Attribute

Syntax

```
<Path>.A.BwEntranceCtrl:method  
<Path>.B.BwEntranceCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyTwoLaneTrack.B.BwEntranceCtrl := &MyBackwardEntranceMethod
```

See also

[Backward Entrance Control](#)

BwEntranceCtrlFront [SimTalk] - lane A or B

Sets if the *TwoLaneTrack* designated by <Path> activates the **Backward Entrance Control** as soon as the **Front** of the *Transporter* enters the *TwoLaneTrack* on the designated lane via its exit and/or when it moves toward its entrance after entering (true) or not (false).

Type

Attribute

Syntax

```
<Path>.A.BwEntranceCtrlFront:boolean  
<Path>.B.BwEntranceCtrlFront:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.B.BwEntranceCtrl := &Method1  
MyTwoLaneTrack.B.BwEntranceCtrlFront := false
```

See also

[Backward Entrance Control](#)

BwEntranceCtrlRear [SimTalk] - lane A or B

Sets if the *TwoLaneTrack* designated by <Path> activates the **Backward Entrance Control** as soon as the **Rear** of the *Transporter* enters the *TwoLaneTrack* via its exit and/or when it moves toward its entrance after entering (true) or not (false).

Type

Attribute

Syntax

```
<Path>.A.BwEntranceCtrlRear: boolean  
<Path>.B.BwEntranceCtrlRear: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.A.BwEntranceCtrl := &MyMethod  
MyTwoLaneTrack.A.BwEntranceCtrlRear := false  
MyTwoLaneTrack.B.BwEntranceCtrl := &MyBackwardEntranceMethod  
MyTwoLaneTrack.B.BwEntranceCtrlRear := true
```

See also

Backward Entrance Control

BwExitCtrl [SimTalk] - lane A or B

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when a *Transporter* exits the designated lane of the *TwoLaneTrack* via its entrance and/or when the *Transporter* moves toward its entrance when exiting.

Type

Attribute

Syntax

```
<Path>.A.BwExitCtrl:method  
<Path>.B.BwExitCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyTwoLaneTrack.B.BwExitCtrl := &MyBackwardExitMethod
```

See also

Backward Exit Control

BwExitCtrlFront [SimTalk] - lane A or B

Sets if the *TwoLaneTrack* designated by <Path> activates the **Backward Exit Control** as soon as the **Front** of the *Transporter* exits the designated lane of the *TwoLaneTrack* via its entrance and/or when the *Transporter* moves toward its entrance when exiting (*true*) or not (*false*).

Type

Attribute

Syntax

```
<Path>.A.BwExitCtrlFront:boolean  
<Path>.B.BwExitCtrlFront:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.B.BwExitCtrl := &MyBackwardExitMethod  
MyTwoLaneTrack.A.BwExitCtrlFront := false
```

See also

[Backward Exit Control](#)

BwExitCtrlRear [SimTalk] - lane A or B

Sets if the *TwoLaneTrack* designated by <Path> activates the **Backward Exit Control** as soon as the **Rear** of the *Transporter* exits the designated lane of the *TwoLaneTrack* via its entrance and/or when the *Transporter* moves toward its entrance when exiting (*true*) or not (*false*).

Type

Attribute

Syntax

```
<Path>.A.BwExitCtrlRear:boolean  
<Path>.B.BwExitCtrlRear:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.B.BwExitCtrl := &MyBackwardExitMethod  
MyTwoLaneTrack.B.BwExitCtrlRear := true
```

See also

[Backward Exit Control](#)

Capacity [SimTalk] - TwoLaneTrack

Sets the maximum number of *Transporters* that may be located on the *TwoLaneTrack* designated by <Path> as a whole or in part at any one time.

Type

Attribute

Syntax

```
<Path>.Capacity:integer
```

Watchable

The attribute is [watchable](#).

Assignment Value

You can assign a value of data type *integer*.

Specify -1 for an infinite **Capacity**.

Example

```
if MyTwoLaneTrack.Capacity = -1  
  @.move(AEConveyor)  
end
```

See also

[Capacity \[text box\] - TwoLaneTrack](#)

DestListA [SimTalk] - TwoLaneTrack

Sets the path to the **Destination List** for *Transporters* which drive in the forward direction on **Lane A** on the *TwoLaneTrack* designated by <Path>.

Remarks

The **Destination List** contains all destination objects, which the *Transporter* can reach while driving in the forward direction on **Lane A** on this *TwoLaneTrack*.

The forward destination list of **Lane A** concurrently is the backward destination list of **Lane B**.

Type

Attribute

Syntax

```
<Path>.DestListA:object
```

Assignment Value

You can assign a value of data type *object*.

Example

```
MyTwoLaneTrack.DestListA := MyDataList1
```

See also

[Destination List A \[text box\]](#)

DestListB [SimTalk] - TwoLaneTrack

Sets the path to the **Destination List** for *Transporters* which drive in the forward direction on **Lane B** on the *TwoLaneTrack* designated by <Path>.

Remarks

The **Destination List** contains all destinations that the *Transporter* can reach while driving in the forward direction on **Lane B** on this *TwoLaneTrack*.

The forward destination list of **Lane B** concurrently is the backward destination list of **Lane A**.

Type

Attribute

Syntax

```
<Path>.DestListB:object
```

Assignment Value

You can assign a value of data type *object*.

Example

```
MyTwoLaneTrack.DestListB := MyDataList2
```

See also

[Destination List B \[text box\]](#)

EntranceCtrl [SimTalk] - lane A or B

Designates a *Method* object of the object designated by <Path>. *Plant Simulation* calls the *Method* as soon as a *Transporter* enters the *TwoLaneTrack* designated by <Path> on the specified lane.

Type

Attribute

Syntax

```
<Path>.A.EntranceCtrl:method  
<Path>.B.EntranceCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyTwoLaneTrack.A.EntranceCtrl := &MyEntranceMethod
```

See also

[Entrance Control \[general description\]](#)

EntranceCtrlFront [SimTalk] - lane A or B

Sets if the *TwoLaneTrack* designated by <Path> triggers the entrance control on the specified lane as soon as the front of the *Transporter* enters the *TwoLaneTrack* (true) or not (false).

Type

Attribute

Syntax

```
<Path>.A.EntranceCtrlFront:boolean  
<Path>.B.EntranceCtrlFront:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.A.EntranceCtrl := &MyEntranceMethod  
MyTwoLaneTrack.A.EntranceCtrlFront := true
```

See also

[Entrance Control \[general description\]](#)

EntranceCtrlRear [SimTalk] - lane A or B

Sets if the *TwoLaneTrack* designated by <Path> triggers the entrance control on the specified lane as soon as the rear of the *Transporter* enters the *TwoLaneTrack* (*true*) or not (*false*).

Type

Attribute

Syntax

```
<Path>.A.EntranceCtrlRear: boolean  
<Path>.B.EntranceCtrlRear: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.A.EntranceCtrl := &MyEntranceMethod  
MyTwoLaneTrack.A.EntranceCtrlRear := true
```

See also

[Entrance Control \[general description\]](#)

EntranceLocked [SimTalk] - lane A or B

Locks the entrance of **Lane A/B** of the *TwoLaneTrack* designated by <Path> (`true`) or `false`). It thus prevents that *Transporters* can enter the object.

Remarks

The *TwoLaneTrack* will finish transporting the *Transporters* already located on it.

It does not accept any additional *Transporters* and enters them into the **Blocking List** of **Lane A/B**.

- If the *Transporter* wants to enter the *TwoLaneTrack* at its entrance, the *Transporter* is entered into the **Forward Blocking List** of **Lane A/B**.
- If the *Transporter* is moving backward and wants to enter a length-oriented object at its exit, the *Transporter* is entered into the **Exit Blocking List** of **Lane A/B**.

The *TwoLaneTrack* starts transporting again, when you set the attribute to `false`. The first *Transporter* in the **Forward Blocking List** of **Lane A/B** is going to be moved on first.

Type

Attribute

Syntax

```
<Path>.A.EntranceLocked: boolean
<Path>.B.EntranceLocked: boolean
```

Watchable

The attribute is [watchable](#).

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.B.EntranceLocked := true
waituntil MyTwoLaneTrack.B.NumMu 10 prio 1
MyTwoLaneTrack.B.EntranceLocked := false
```


See also

[Entrance Locked \[lane A\]](#)

[Entrance Locked \[lane B\]](#)

[Forward Blocking List](#)

[Exit Blocking List](#)

ExitCtrl [SimTalk] - lane A or B

Designates a *Method* object of the object designated by <Path>. *Plant Simulation* calls the *Method* as soon as a *Transporter* exits the *TwoLaneTrack* designated by <Path> on the specified lane

Remarks

If you did not specify a *Method*, the attribute is VOID.

Type

Attribute

Syntax

```
<Path>.A.ExitCtrl:method  
<Path>.B.ExitCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyTwoLaneTrack.A.ExitCtrl := &MyExitMethod
```

See also

[Exit Control \[general description\]](#)

ExitCtrlFront [SimTalk] - lane A or B

Sets if the *TwoLaneTrack* designated by <Path> triggers the **Exit Control** as soon as the front of the *Transporter* exits on the specified lane (*true*) or not (*false*).

Type

Attribute

Syntax

```
<Path>.A.ExitCtrlFront:boolean  
<Path>.B.ExitCtrlFront:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.A.ExitCtrl := &MyExitMethod  
MyTwoLaneTrack.A.ExitCtrlFront := false
```

See also

[Exit Control \[general description\]](#)

ExitCtrlRear [SimTalk] - lane A or B

Sets if the *TwoLaneTrack* designated by <Path> triggers the **Exit Control** as soon as the rear of the *Transporter* exits on the specified lane (*true*) or not (*false*).

Type

Attribute

Syntax

```
<Path>.A.ExitCtrlRear:boolean  
<Path>.B.ExitCtrlRear:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.A.ExitCtrl := &MyExitMethod  
MyTwoLaneTrack.A.ExitCtrlRear := true
```

See also

[Exit Control \[general description\]](#)

ExitLocked [SimTalk] - lane A or B

Locks the exit of **Lane A/B** of the *TwoLaneTrack* designated by <Path> (`true`) or not (`false`). It thus prevents that *Transporters* leave the object.

Remarks

The *TwoLaneTrack* does not transport the *Transporters* to the successor in the material flow, but enters them into the **Exit Blocking List** of **Lane A/B** of the *TwoLaneTrack*.

Once the exit is unlocked again, the *TwoLaneTrack* moves the first *Transporter* in the **Exit Blocking List** of **Lane A/B** on to its successor.

When setting *ExitLocked* to `false` *Transporters* entered into the **Exit Blocking List** will make another attempt to exit the *TwoLaneTrack*.

Note

The *Transporter* does not trigger the **Exit Control** if you set *ExitLocked* to `true`.

Type

Attribute

Syntax

```
<Path>.A.ExitLocked:boolean  
<Path>.B.ExitLocked:boolean
```

Watchable

The attribute is [watchable](#).

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTwoLaneTrack.A.ExitLocked := true
```

See also

[Exit Locked \[lane A\]](#)

ExitLocked [SimTalk] - lane A or B

Exit locked [lane B] of lane B

Exit Blocking List

ExitLocked [SimTalk] - lane A or B

Length [SimTalk] - lane A or B

Sets the physical length of the specified lane of the *TwoLaneTrack* designated by <Path>.

Remarks

A *Transporter* enters the specified lane of the *TwoLaneTrack* at position 0 and exits after covering the length you enter here. *Speed* and *Length* yield the dwelling time on the *TwoLaneTrack*.

Type

Attribute

Syntax

```
<Path>.A.Length: length  
<Path>.B.Length: length
```

Watchable

The attribute is [watchable](#).

Assignment Value

You can assign a value of data type *length*.

Example

```
MyTwoLaneTrack.A.Length := 44    // meters  
MyTwoLaneTrack.B.Length := 43.8  // meters
```

SimTalk

[OccupiedLength \[SimTalk\] - lane A or B](#)

See also

[Length \[text box\] - lane A](#)

[Length \[text box\] - lane B](#)

PullCtrl [SimTalk] - lane A or B

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when the *TwoLaneTrack* designated by <Path> is ready to accept a new *Transporter* on the specified lane or when it is ready, when a new *Transporter* is **Waiting** at its entrance.

In the **Pull Control** you can determine which *Transporter*, which intends to move onto the *TwoLaneTrack*, the *TwoLaneTrack* will accept. For this you have to get the **Forward Blocking List** of the respective lane with the method *fwBlockList* and unblock a *Transporter* in the list with the method *unblock*.

Note

If you only need the first entry of the **Forward Blocking List**, use the read-only-attribute *FwBlockListEntry1* for those objects which have a **Forward Blocking List**. Access is much faster than with the method *fwBlockList*.

Suppose that red parts are especially important and thus have to be processed first. You can always pull red parts from the predecessors in the **Pull Control** first and then parts with other colors.

Note

The **Pull Control** does not determine if a *Transporter* is to be moved on to this successor or to another successor, but it selects one of the *Transporters* which already decided to move to this successor.

Note

For the *TwoLaneTrack* the **Pull Control** only applies to *Transporters* that drive forward, meaning that they enter the *TwoLaneTrack* at the entrance and they exit at the exit. It does not apply to *Transporters* driving in reverse entering at the exit and not to *Transporters* that have been moved to a certain position.

Type

Attribute

Syntax

```
<Path>.A.PullCtrl:method  
<Path>.B.PullCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyTwoLaneTrack.A.PullCtrl := &MyPullMethod
```

SimTalk

[fwBlockList \[SimTalk\] - lane A or B](#)

[FwBlockListEntry1 \[SimTalk\]](#)

[DecimalPlaces \[SimTalk\] - Variable](#)

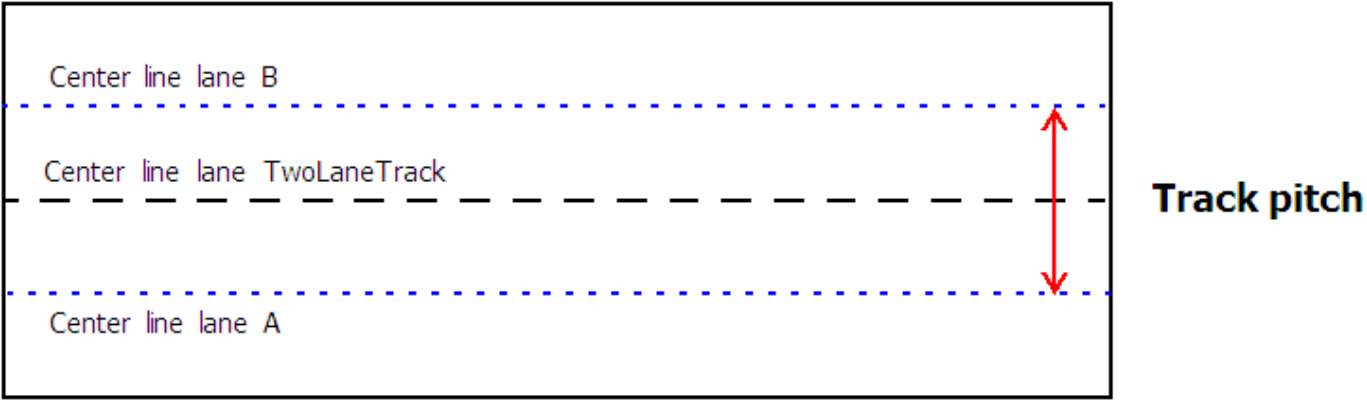
See also

[Pull Control \[general description\]](#)

[Waiting \[state, material flow objects\]](#)

TrackPitch [SimTalk]

Sets the **Track Pitch**, which is the distance between the center lines of the two lanes of the *TwoLaneTrack* designated by <Path>.



Compare this example:

Track pitch 1	Track pitch 0.6
	

Type

Attribute

Syntax

```
<Path>.TrackPitch:length
```

Assignment Value

You can assign a value of data type *length*.

Example

```
MyTwoLaneTrack.TrackPitch := 5
```

See also

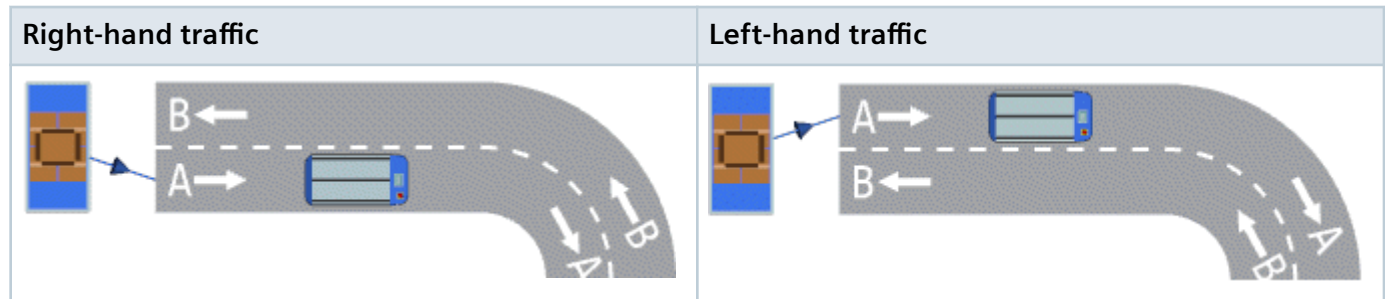
[Track Pitch \[text box\]](#)

Traffic [SimTalk] - TwoLaneTrack

Sets the side of the *TwoLaneTrack* designated by <Path> on which the bidirectional traffic moves.

Remarks

Traffic on a *TwoLaneTrack* looks like this:



Type

Attribute

Syntax

```
<Path>.Traffic:string
```

Assignment Value

You can assign a value of data type *string*.

You can specify "Right-hand traffic" or "Left-hand traffic".

Example

```
MyTwoLaneTrack.traffic := "left-hand traffic"
```

See also

[Right-hand Traffic/Left-hand Traffic](#)

Width [SimTalk] - TwoLaneTrack

Sets the width of the *TwoLaneTrack* designated by <Path>.

Type

Attribute

Syntax

```
<Path>.Width:length
```

Watchable

The attribute is [watchable](#).

Assignment Value

You can assign a value of data type *length*.

Note

In *SimTalk* 2.0 you can specify the length units m, mm, km, cm, yd, ft, and in.

Type in the unit directly after the values, without a separating blank space, for example 10m or 10.2m.

You can specify the unit for *floating point values* and for *integer values*.

Example

```
MyTwoLaneTrack.Width := 4
```

See also

[Width \[text box\] - TwoLaneTrack](#)

FlowControl

FlowControl

Use the object *FlowControl* for modeling strategies for splitting-up and bringing together the flow of materials in your plant.

Image



Description

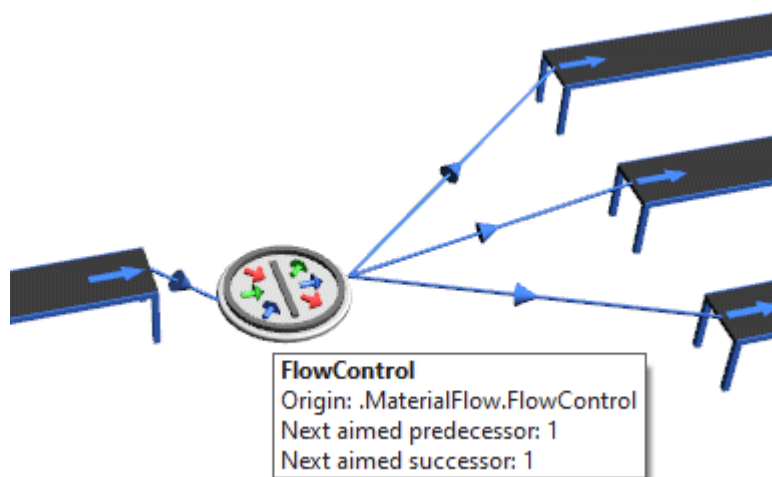
The *FlowControl* does not process the *MUs*, it only distributes them among the stations that succeed it in the sequence of stations in the simulation model.

Insert the *FlowControl* between at least two other objects to control the flow of materials between these objects. If need be, you can also combine several *FlowControl* objects. The number of predecessors and successors is unlimited.

Note

You cannot model cycles, meaning that may not connect a *FlowControl* with itself, not even via other *FlowControls* or *Interfaces*. You can, for example **not** model like this: *FlowControl1* -> *Frame.Interface1* -> *Frame.FlowControl2* -> *Frame.Interface2* -> *FlowControl1*.

To show a **tooltip** with information about the *FlowControl*, hover with the mouse over it.



To change the length of the graphic and the anchor points of the *FlowControl*, click **Show Manipulators**



on the **Edit** ribbon tab or press **M** on the keyboard.

Add the Object to the Simulation Model

To add the object *FlowControl* to your simulation model, click  **Manage Class Library** > **Basic Objects** > **MaterialFlow** > **FlowControl** on the **Home** ribbon tab.